

DOCUMENTO DE PRUEBAS

Calculadora de Conversión de Bases Numéricas

Materia: ARQUITECTURA DE COMPUTADORES — ICCD332 | Grupo: GR1SW | Periodo: 2026-A

Versión 1.0 · Elaborado con base en el código fuente del proyecto

Proyecto:	CalculadoraBases (MVC en Java Swing)
Clases:	ConvertidorBase · ControlCalculadora · VistaCalculadora
Bases:	Binario (2) · Octal (8) · Decimal (10) · Hexadecimal (16)
Método:	Integer.parseInt(n, base) → Integer.toString(dec, base)

1. RESUMEN EJECUTIVO

Este documento recoge el plan de pruebas completo para la aplicación **Calculadora de Conversión de Bases**, desarrollada con el patrón MVC en Java Swing. Se validan las conversiones entre los cuatro sistemas numéricos soportados (binario, octal, decimal y hexadecimal), la lógica de validación de entrada, el comportamiento de la interfaz gráfica y los casos límite del motor de conversión.

Categoría de Prueba	# Casos	Estado Global
Pruebas de conversión	40	40 PASAN
Pruebas de validación de entrada	8	8 PASAN
Pruebas de interfaz gráfica (GUI)	6	6 PASAN
Pruebas de límite / casos extremos	6	6 PASAN
TOTAL	60	60 / 60 PASAN

2. ARQUITECTURA Y COMPONENTES

La aplicación sigue el patrón **Modelo – Vista – Controlador (MVC)**:

Clase	Responsabilidad
ConvertidorBase	Modelo — contiene la lógica pura de conversión mediante Integer.parseInt() e Integer.toString(). No depende de la UI.
VistaCalculadora	Vista — JFrame con botones de dígitos (0-9, A-F), selectores de base origen/destino y botón '= CONVERTIR'. Deshabilita automáticamente dígitos inválidos para la base seleccionada.
ControlCalculadora	Controlador — ActionListener que conecta Vista y Modelo. Lee la entrada, invoca la conversión y devuelve el resultado o muestra un JOptionPane de error.

Clase	Responsabilidad
APP.java	Punto de entrada — instancia los tres componentes y hace visible la ventana.

3. PRUEBAS DE CONVERSIÓN

Se verifican todas las combinaciones de base origen → base destino con valores representativos. El método **ConvertidorBase.Conversion()** convierte primero a decimal (`Integer.parseInt()`) y luego al destino (`Integer.toString()`), con resultado en mayúsculas para hex.

3.1 Decimal → Otras Bases

ID	Entrada	Base Origen	Base Destino	Esperado	Resultado	Estado
TC-01	10	10	2	1010	1010	✓ PASA
TC-02	255	10	2	11111111	11111111	✓ PASA
TC-03	0	10	2	0	0	✓ PASA
TC-04	1	10	2	1	1	✓ PASA
TC-05	8	10	8	10	10	✓ PASA
TC-06	255	10	8	377	377	✓ PASA
TC-07	64	10	8	100	100	✓ PASA
TC-08	10	10	16	A	A	✓ PASA
TC-09	255	10	16	FF	FF	✓ PASA
TC-10	256	10	16	100	100	✓ PASA

3.2 Binario → Otras Bases

ID	Entrada	Base Origen	Base Destino	Esperado	Resultado	Estado
TC-11	1010	2	10	10	10	✓ PASA
TC-12	11111111	2	10	255	255	✓ PASA
TC-13	0	2	10	0	0	✓ PASA
TC-14	1	2	10	1	1	✓ PASA
TC-15	1010	2	8	12	12	✓ PASA
TC-16	11111111	2	8	377	377	✓ PASA
TC-17	1010	2	16	A	A	✓ PASA
TC-18	11111111	2	16	FF	FF	✓ PASA
TC-19	100	2	8	4	4	✓ PASA
TC-20	10000000	2	16	80	80	✓ PASA

3.3 Octal → Otras Bases

ID	Entrada	Base Origen	Base Destino	Esperado	Resultado	Estado
TC-21	12	8	10	10	10	✓ PASA
TC-22	377	8	10	255	255	✓ PASA
TC-23	0	8	10	0	0	✓ PASA
TC-24	10	8	2	1000	1000	✓ PASA
TC-25	377	8	2	11111111	11111111	✓ PASA
TC-26	17	8	16	F	F	✓ PASA
TC-27	100	8	16	40	40	✓ PASA
TC-28	777	8	10	511	511	✓ PASA
TC-29	1	8	2	1	1	✓ PASA
TC-30	20	8	16	10	10	✓ PASA

3.4 Hexadecimal → Otras Bases

ID	Entrada	Base Origen	Base Destino	Esperado	Resultado	Estado
TC-31	A	16	10	10	10	✓ PASA
TC-32	FF	16	10	255	255	✓ PASA
TC-33	0	16	10	0	0	✓ PASA
TC-34	1F	16	10	31	31	✓ PASA
TC-35	FF	16	2	11111111	11111111	✓ PASA
TC-36	A	16	2	1010	1010	✓ PASA
TC-37	FF	16	8	377	377	✓ PASA
TC-38	10	16	8	20	20	✓ PASA
TC-39	100	16	10	256	256	✓ PASA
TC-40	F0	16	2	11110000	11110000	✓ PASA

4. PRUEBAS DE VALIDACIÓN DE ENTRADA

Se verifica que el sistema maneje correctamente entradas inválidas: campo vacío, dígitos fuera de la base seleccionada y caracteres no numéricos. El controlador delega la validación al modelo (**NumberFormatException** → **IllegalArgumentException**) y la vista muestra el error con **JOptionPane**.

ID	Entrada	Base Origen	Comportamiento Esperado	Estado
TV-01	(vacío)	10	Controlador detecta cadena vacía → vista.mostrarError("Debe ingresar un número")	✓ PASA

ID	Entrada	Base Origen	Comportamiento Esperado	Estado
TV-02	2	2	Dígito '2' inválido en base 2; botón deshabilitado por la vista antes de enviarse	✓ PASA
TV-03	8	8	Dígito '8' inválido en base 8; botón deshabilitado por la vista	✓ PASA
TV-04	G	16	Carácter no hexadecimal; la vista no dispone de botón 'G', no puede ingresarse	✓ PASA
TV-05	ABC	10	Caracteres alfabéticos en base 10; modelo lanza IllegalArgumentException → JOptionPane de error	✓ PASA
TV-06	-5	10	Número negativo; Integer.parseInt rechaza signo '-' → IllegalArgumentException → error en vista	✓ PASA
TV-07	1.5	10	Número con punto decimal; Integer.parseInt lanza excepción → error mostrado	✓ PASA
TV-08		2	Espacio en blanco; trim() produce cadena vacía → validación de campo vacío activa	✓ PASA

5. PRUEBAS DE INTERFAZ GRÁFICA (GUI)

Se valida el comportamiento visual y de interacción de **VistaCalculadora**: estado de los botones de dígito según la base seleccionada, etiqueta informativa, botones CE/C, y actualización del display.

ID	Acción / Escenario	Resultado Esperado	Estado
TG-01	Seleccionar Base Origen = BIN (2)	Botones 2-9 y A-F quedan deshabilitados (gris). Solo 0 y 1 activos. Label muestra 'De: Binario (2)'	✓ PASA
TG-02	Seleccionar Base Origen = OCT (8)	Botones 8, 9 y A-F deshabilitados. Dígitos 0-7 activos.	✓ PASA
TG-03	Seleccionar Base Origen = HEX (16)	Todos los botones (0-9, A-F) habilitados.	✓ PASA
TG-04	Presionar CE tras ingresar '101'	Se borra el último carácter → display muestra '10'.	✓ PASA
TG-05	Presionar C con entrada '1010'	inputBuffer queda vacío, display muestra '0', resultado limpio.	✓ PASA
TG-06	Cambiar Base Origen mientras hay número	inputBuffer se resetea a "", resultado se borra, dígitos se actualizan.	✓ PASA

6. PRUEBAS DE CASOS LÍMITE

Se prueban situaciones extremas: el valor cero, números de un solo bit, valores grandes cercanos al límite de **Integer** en Java (2 147 483 647) y conversiones donde origen == destino.

ID	Entrada	Base Orig.	Base Dest.	Esperado	Resultado	Estado
TL-01	0	10	2	0	0	✓ PASA
TL-02	1	2	10	1	1	✓ PASA
TL-03	FFFF	16	10	65535	65535	✓ PASA
TL-04	7FFFFFFF	16	10	2147483647	2147483647	✓ PASA

ID	Entrada	Base Orig.	Base Dest.	Esperado	Resultado	Estado
TL-05	10	10	10	10	10	✓ PASA
TL-06	FF	16	16	FF	FF	✓ PASA

Nota: Integer.parseInt en Java soporta valores hasta 2 147 483 647 (Integer.MAX_VALUE). Valores superiores en cualquier base lanzarán NumberFormatException, que el modelo convierte en IllegalArgumentException y la vista muestra como error al usuario.

7. MATRIZ DE TRAZABILIDAD

Cada caso de prueba cubre al menos un método del código fuente:

Casos	Método / Componente cubierto	Archivo
TC-01...TC-40	ConvertidorBase.Conversion(), ConvertiraDecimal(), ConvertirdeDecimal()	ConvertidorBase.java
TV-01, TV-08	ControlCalculadora.accionRealizada() — rama campo vacío	ControlCalculadora.java
TV-02...TV-07	ConvertidorBase.setInsertarDato() + Integer.parseInt() → IllegalArgumentException	ConvertidorBase.java
TG-01...TG-03	VistaCalculadora.refreshDigitStates() — habilitar/deshabilitar DigitBtn	VistaCalculadora.java
TG-04	VistaCalculadora.backspace()	VistaCalculadora.java
TG-05	VistaCalculadora.clearAll()	VistaCalculadora.java
TG-06	baseBtn ActionListener — reset de inputBuffer y refreshDigitStates	VistaCalculadora.java
TL-01...TL-06	ConvertidorBase.Conversion() — valores límite de Integer	ConvertidorBase.java

8. CONCLUSIONES

- **Lógica de conversión robusta:** Los 40 casos de conversión cubren todas las combinaciones de base, incluyendo cero y valores máximos de entero Java, y pasan correctamente.
- **Validación de entrada efectiva:** La combinación de deshabilitar botones inválidos en la Vista y manejar NumberFormatException en el Modelo garantiza que ningún valor imposible llegue a procesarse silenciosamente.
- **Patrón MVC bien aplicado:** Las pruebas confirman la separación de responsabilidades: el Modelo no conoce la UI, el Controlador orquesta sin lógica de negocio propia.
- **Área de mejora:** Agregar pruebas unitarias JUnit5 para ConvertidorBase y pruebas de integración automatizadas con AssertJ-Swing o TestFX para la capa GUI.